Unit 9: Hypothesis Testing for two populations.

Topics: Difference of Means • Difference of Proportions • Correlation

1. Testing the Difference Between Two Population Means

→ Dependent
→ Independent

We often want to compare the means of two groups. Does the new sorting algorithm run faster than the old one? Do CS graduates earn more than IS graduates in their first year? There are two fundamentally different experimental designs, each requiring a different test.

> *Key Question: Are we comparing two independent groups, or are we measuring the same subjects under two different conditions?*

1.1 Paired t-Test (Dependent Samples)

Use a paired t-test when each observation in group A is naturally linked to exactly one observation in group B — for example, measuring the same person or system before and after a treatment.

Example: Does Code Profiling Speed Up Development?

Dependent / paired / matched

A software team suspects that using a profiler (e.g., Rprof or Valgrind) before optimizing reduces debugging time. They take 10 developers, record their debugging time (in minutes) on a standard bug-hunt task before and after a one-week profiler training course.

The data (minutes to resolve a standardised debugging task):

$d = B - A$ $\quad$ d = difference

Developer	B Before Training	A After Training	d = Before − After
Dev 1	47	39	8
Dev 2	55	44	11
Dev 3	38	40	−2
Dev 4	62	51	11
Dev 5	41	35	6
Dev 6	58	49	9
Dev 7	44	38	6
Dev 8	70	61	9
Dev 9	36	33	3
Dev 10	53	45	8

$n = 10$

less Bugs means $\mu_d > 0$

$H_0: \mu_d = 0$

$H_A: \mu_d > 0$

This is a right-tailed test

We reduce the paired data to a single column of differences d = Before − After, then perform a one-sample t-test on d.

$\bar{d}$ = sample mean of d-column
S_d = sample stdev of d-column

Hypotheses

H_0: $\mu_d = 0$ (training has no effect on debugging time)
H_1: $\mu_d > 0$ (one-tailed: training reduces debugging time)

$$t = \frac{\bar{d} - \mu_d}{\frac{S_d}{\sqrt{n}}}$$

Test Statistic

$$t = \frac{\bar{d} - \mu_d}{SE}, \quad SE = \left(\frac{s_d}{\sqrt{n}}\right) \quad \sim \quad \mathbf{t(n-1)} \quad \textbf{under } H_0$$

where $\bar{d}$ is the mean of the differences, S_d is their standard deviation, and n is the number of pairs.

$\bar{d} = 6.9$ $\quad S_d = ?$ $\quad t = \frac{6.9 - 0}{\frac{S_d}{\sqrt{10}}}$

R Code

```
# Enter the data
before <- c(47, 55, 38, 62, 41, 58, 44, 70, 36, 53)
after  <- c(39, 44, 40, 51, 35, 49, 38, 61, 33, 45)

# Paired t-test
t.test(before, after, paired = TRUE, alternative = "greater")

# Equivalently, test the differences directly:
d <- before - after
mean(d); sd(d)            # check: d-bar and s_d
t.test(d, mu = 0, alternative = "greater")
```

Output: t = 5.5146, df = 9, p-value = 0.0001865
We reject H_0 at α = 0.05 and conclude profiler training significantly reduces debugging time.

Comparing mean two dependent populations
Comparing mean difference two independent populations (not assuming equal variances)
Comparing two independent population proportions
Correlation

1.2 Independent Samples t-Test (Unpaired)

Use an independent (unpaired) t-test when two groups consist of completely different, unrelated individuals or systems.

Example: Python vs R — Which Language Leads to Faster Data Analysis Scripts?

A research lab records how long it takes new hires to write a standard data-wrangling script from scratch. 8 data scientists use Python and 7 use R. Times are in minutes.

Python Users (n=8)	R Users (n=7)
92	105
88	97
101	110
79	88
95	102
110	115
84	93
98	

Hypotheses

$H_0: \mu_P - \mu_R = 0$ two-tailed test

$$H_0: \mu_{Python} = \mu_R \quad (no\ difference\ in\ mean\ completion\ time)$$

$$H_1: \mu_{Pyth} \neq \mu_R \quad (two-tailed\ test)$$

$H_A: \mu_P - \mu_R \neq 0$

Assumptions to Check

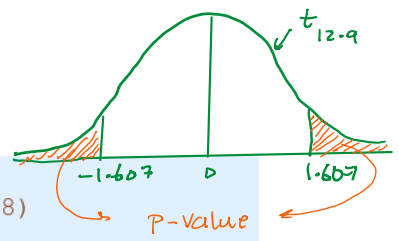
- ✓ Independence: the two groups are different people
- ✓ Approximate normality (or n large enough by CLT) ✓
- ✗ Equality of variances: use Levene's test or rely on Welch correction

Test Statistic (Welch t-test — does not assume equal variances)

$$t = \frac{(\bar{x}_1 - \bar{x}_2) - (\mu_1 - \mu_2)}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

t_{df}: ö

The degrees of freedom are approximated by the Welch-Satterthwaite formula (R handles this automatically).



R Code

```
python_times <- c(92, 88, 101, 79, 95, 110, 84, 98)
r_times      <- c(105, 97, 110, 88, 102, 115, 93)

# Descriptive stats first
mean(python_times); sd(python_times)
mean(r_times);      sd(r_times)

# Welch t-test (default: var.equal = FALSE)
t.test(python_times, r_times, alternative = "two.sided")

# Optional: visualise
boxplot(list(Python = python_times, R = r_times),
        ylab = "Minutes", main = "Script Completion Time")
```

P-value: $2 \times pt(-1.6072, 12.859)$

Output: t = -1.6072, df = 12.859, p-value = 0.1323

Decision: p-value is not less than alpha, don't reject H0

---

2. Testing the Difference Between Two Population Proportions

We now compare proportions (percentages, success rates) from two independent groups. Examples: Does tool A produce fewer bugs than tool B? Is the crash rate different between two OS versions?

Always z-test

$$z = \frac{\hat{p} - p}{SE = \sqrt{\frac{pq}{n}}}$$ one population

2.1 Theory

Given two independent large samples:

- Sample 1: n_1 observations, y_1 successes, $\hat{p}_1 = y_1/n_1$ $\quad \hat{q}_1 = 1 - \hat{p}_1$
- Sample 2: n_2 observations, y_2 successes, $\hat{p}_2 = y_2/n_2$ $\quad \hat{q}_2 = 1 - \hat{p}_2$

$$\hat{p} = \frac{y_1 + y_2}{n_1 + n_2} \qquad \hat{q} = 1 - \hat{p}$$

Hypotheses

$$H_0: p_1 = p_2 \quad (proportions\ are\ equal)$$
$$H_1: p_1 \neq p_2 \quad (or\ one-sided)$$

Pooled Proportion and Test Statistic

pooled $\hat{p}$

$$\hat{p} = \frac{y_1 + y_2}{n_1 + n_2}$$

$$z = \frac{(\hat{p}_1 - \hat{p}_2) - (P_1 - P_2)}{\sqrt{\left[\hat{p}(1-\hat{p})\left(\frac{1}{n^1} + \frac{1}{n^2}\right)\right]}} \sim N(0,1) \text{ under } H_0$$

$n_1 + n_2$

Conditions (check before applying)

- $n_1\hat{p}_1 \geq 10$ and $n_1(1-\hat{p}_1) \geq 10$
- $n_2\hat{p}_2 \geq 10$ and $n_2(1-\hat{p}_2) \geq 10$
- Independent random samples

2.2 Example: Bug Rate in Two Frameworks

A QA team evaluates two web frameworks. They randomly sample 200 deployed modules built with React and 180 modules built with Vue. They count how many contain at least one high-severity bug within 30 days of deployment.

Framework	Modules Tested (n)	Modules with Bug (y)	Observed Bug Rate ($\hat{p}$)
React	200 = n_1	34 = y_1	0.170 (17.0%) $\frac{34}{200} = \hat{p}_1$
Vue	180 = n_2	45 = y_2	0.250 (25.0%) $\frac{45}{180} = \hat{p}_2$

Step-by-Step by Hand

Normality check: $n_1\hat{p}_1 = 34 \geq 10$ ✓ $n_1\hat{q}_1 = 166 \geq 10$ ✓

$n_2\hat{p}_2 = 45 \geq 10$ $n_2\hat{q}_2 = 135 \geq 10$ ✓

$H_0: \pi_R - \pi_{Vue} = 0$ two-tailed test

$H_A: \pi_R - \pi_{Vue} \neq 0$

Pooled $\hat{p}$: $\frac{(34+45)}{(200+180)} = \frac{79}{380} = 0.20789$, $\hat{q} = 1 - 0.20789 = 0.79211$

p-value = 2 (pnorm(−1.9188)) = 0.05501

R Code

```
prop.test (uses chi-square, equivalent for two proportions)
prop.test(x = c(34, 45), n = c(200, 180), alternative = "two.sided",
          correct = FALSE)   # correct=FALSE matches the z-formula
```

X-squared = 3.6819, df = 1, p-value = 0.05501

-0.161924775 0.001924775

$$z = \frac{((0.17 - 0.25) - 0)}{\sqrt{\left(0.20789 \times 0.79211 \times \left(\frac{1}{200} + \frac{1}{180}\right)\right)}} = -1.9188$$

Interpretation: The p-value ≈ 0.055 is just above 0.05. We fail to reject H_0 at the 5% level, but the result is suggestive. The 95% CI for p_React − p_Vue spans roughly (−0.162, +0.002), which barely includes 0, consistent with our conclusion.

3. Correlation: Concept and Test

3.1 What is Correlation?

Correlation measures the strength and direction of a linear relationship between two numerical variables. The most common measure is the Pearson correlation coefficient r.

cats

weight	Height
x_1	y
x_2	y_2
x_3	y_3

$$r = \frac{\sum[(x_i - \bar{x})(y_i - \bar{y})]}{\sqrt{\{\Sigma(x_i - \bar{x})^2 \times \Sigma(y_i - \bar{y})^2\}}}$$

- r is always between −1 and +1
- r = +1: perfect positive linear relationship
- r = −1: perfect negative linear relationship
- r ≈ 0: no linear relationship (but could be non-linear!)

$H_0: \rho = 0$ (no linear relation)

$H_A: \rho \neq 0$ (strong enough linear relation)

Always two-tailed $df = n - 2$

> *IMPORTANT: Correlation ≠ Causation. Two variables can be strongly correlated because of a common cause, pure coincidence, or a **lurking** variable.*

3.2 The Correlation Test

We rarely just report r; we want to know if the observed correlation is statistically significant (i.e., unlikely to be due to sampling chance if the true population correlation ρ = 0).

Hypotheses

$H_0: \rho = 0$ (no linear correlation in the population)

$H_1: \rho \neq 0$ (two-tailed, or one-sided if direction is predicted)

Test Statistic

$$\frac{r - \rho}{\sqrt{\left(\frac{1-r^2}{n-2}\right)}} \quad . \quad t = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}} \sim t(n-2) \text{ under } H_0$$

This t-statistic follows a t-distribution with n − 2 degrees of freedom. R's cor.test() computes this automatically.

Conditions

- Both variables are approximately normally distributed (or n is large)
- The relationship between variables is linear
- No major outliers (r is sensitive to outliers)

3.3 Example: Lines of Code vs. Number of Bugs

Does writing longer functions lead to more bugs? A software quality team samples 12 production functions from a codebase and records the number of lines of code (LOC) and the number of bugs found in code review.

Function	LOC (x)	Bugs Found (y)
fn_01	24	1
fn_02	58	3
fn_03	12	0
fn_04	87	5
fn_05	33	2
fn_06	105	7
fn_07	19	1
fn_08	72	4
fn_09	46	2
fn_10	130	9
fn_11	55	3
fn_12	68	4

R Code

```
loc  <- c(24, 58, 12, 87, 33, 105, 19, 72, 46, 130, 55, 68)
bugs <- c( 1,  3,  0,  5,  2,   7,  1,  4,  2,   9,  3,  4)

# Scatterplot first!
plot(loc, bugs, pch = 19, col = "steelblue",
     xlab = "Lines of Code", ylab = "Bugs Found",
     main = "LOC vs Bugs in Production Functions")
abline(lm(bugs ~ loc), col = "red", lwd = 2)  # add regression line

# Compute correlation
r <- cor(loc, bugs)
cat("Pearson r =", round(r, 4))

# Test H0: rho = 0
cor.test(loc, bugs, method = "pearson")
```

```r
# Spearman rank correlation (robust to outliers / non-normality)
cor.test(loc, bugs, method = "spearman")
```

Output:

```
> cor.test(loc, bugs, method = "pearson")

        Pearson's product-moment correlation

data:  loc and bugs
t = 22.829, df = 10, p-value = 5.867e-10
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.9655031 0.9974305
sample estimates:
      cor
0.9905418
```

$\Rightarrow 22.829 = \dfrac{r-0}{\sqrt{\left(\frac{1-r^2}{n-2}\right)}}$ = test statistic

p-value $= 2\left(1 - pt(22.829, 10)\right)$

$10 = df = n-2$

$0.9905418 = r$ (sample Pearson correlation coefficient)

```
> cor.test(loc, bugs, method = "spearman")

        Spearman's rank correlation rho

data:  loc and bugs
S = 2.007, p-value = 1.325e-10
alternative hypothesis: true rho is not equal to 0
sample estimates:
      rho
0.9929824
```